In [7]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
import mlflow
```

In [8]:

```
from regression_models import LinearRegression, Lasso, Ridge, NoRegularization
# #Class
# class LinearRegression(object):

#     kfold = KFold(n_splits=5)

#     #init
#     def __init__(self, regularization, lr=0.001, method='batch', num_epochs=500, bs = 50, cv=kfold, initialise_method='zeros', use_momentum=False, momentum=0.9):
#         self.lr         = lr
#         self.num_epochs = num_epochs
#         self.bs         = bs
#         self.method     = method
#         self.cv         = cv
#         self.regularization = regularization
#         self.initialise_method = initialise_method
#         self.use_momentum = use_momentum
#         self.momentum = momentum

#     #mse
#     def mse(self, ytrue, ypred):
#         #ytrue shape:  (m, )  ==> m = number of samples
#         return ((ypred - ytrue) ** 2).sum() / ytrue.shape[0]

#     #r2
#     def r2(self, ytrue, ypred):
#         ss_res = ((ytrue - ypred) ** 2).sum()
#         ss_tot = ((ytrue - ytrue.mean()) ** 2).sum()
#         return 1 - (ss_res / ss_tot)

#     # weight initialisation
#     def weight_initialise(self, n_features):
#         if self.initialise_method == "xavier":
#             lower, upper = -(1.0 / np.sqrt(n_features)), (1.0 / np.sqrt(n_features))
#             return np.random.uniform(lower, upper, size=n_features)
#         else:
#             return np.zeros(n_features)

#     #fit
#     def fit(self, X_train, y_train):

#         #create a list for keeping kfold scores
#         self.kfold_scores = list()
#         self.kfold_r2_scores = list()

#         #variable to know our loss is not improving anymore
#         #if the new loss !< old loss,  we stop!  (0.01 -> tolerance - tol)
#         self.val_loss_old = np.inf

#         #cross validation
#         for fold, (train_idx, val_idx) in enumerate(self.cv.split(X_train)):

#             X_cross_train = X_train[train_idx]
#             y_cross_train = y_train[train_idx]
#             X_cross_val   = X_train[val_idx]
#             y_cross_val   = y_train[val_idx]

#             self.theta = self.weight_initialise(X_cross_train.shape[1])
#             self.prev_step = np.zeros(X_cross_train.shape[1])

#             for epoch in range(self.num_epochs):

#                 #Shuffle the data a little so that order does not impact our model
#                 perm = np.random.permutation(X_cross_train.shape[0]) #perm = [2 50 67 1 .... ]

#                 X_cross_train = X_cross_train[perm]
#                 y_cross_train = y_cross_train[perm]

#                 if self.method == 'sto':
#                     for i in range(X_cross_train.shape[0]):
#                         X_method_train = X_cross_train[i:i+1, :]
#                         y_method_train = y_cross_train[i:i+1]
#                         train_loss = self._train(X_method_train, y_method_train)

#                 elif self.method == 'mini':
#                     for batch_idx in range(0, X_cross_train.shape[0], self.bs):
#                         X_method_train = X_cross_train[batch_idx:batch_idx+self.bs, :]
#                         y_method_train = y_cross_train[batch_idx:batch_idx+self.bs]
#                         train_loss = self._train(X_method_train, y_method_train)

#                 else:
#                     X_method_train = X_cross_train
#                     y_method_train = y_cross_train
#                     train_loss = self._train(X_method_train, y_method_train)

#                 yhat_val = self.predict(X_cross_val)
#                 val_loss_new = self.mse(y_cross_val, yhat_val)
#                 val_r2 = self.r2(y_cross_val, yhat_val)

#                 #early stopping
#                 if np.allclose(val_loss_new, self.val_loss_old):
#                     break
#                 self.val_loss_old = val_loss_new

#             self.kfold_scores.append(val_loss_new)
#             self.kfold_r2_scores.append(val_r2)

#             print(f"Fold {fold}: MSE={val_loss_new}, R2={val_r2}")


#     #train
#     def _train(self, X, y):
#         #X shape: (m, n)
#         #y shape: (m, )
#         #theta shape: (n, )

#         #1. predict
#         yhat = self.predict(X)

#         #2. grad
#         m = X.shape[0]
#         grad = (1/m) * X.T @ (yhat - y) + self.regularization.derivation(self.theta)

#         # (n, m) @ (m, ) - (m, ) = (m, ) ===> (n, )

#         #3. update
#         step = self.lr * grad
#         if self.use_momentum:
#             self.theta = self.theta - step + self.momentum * self.prev_step
#         else:
#             self.theta = self.theta - step
#         self.prev_step = step

#         #return
#         return self.mse(y, yhat)


#     #predict
#     def predict(self, X):
#         return X @ self.theta  #(m, n) @ (n, ) = (m, )  <===== y

#     #get theta
#     def _coef(self):
#         return self.theta[1:]

#     #get bias
#     def _bias(self):
#         return self.theta[0]

#     def plot_feature_importance(self, feature_names=None, top_n=None):
#         coefs = self._coef()
#         if feature_names is None:
#             feature_names = [f'feature_{i}' for i in range(len(coefs))]
```

```
#         importance = np.abs(coefs)
#         order = np.argsort(importance)  # ascending -> nicer barh order

#         if top_n is not None:
#             order = order[-top_n:]

#         plt.figure(figsize=(8, max(4, len(order) * 0.3)))
#         plt.barh(np.array(feature_names)[order], importance[order])
#         plt.xlabel('|coefficient|')
#         plt.title('Feature importance (by coefficient magnitude)')
#         plt.tight_layout()
#         plt.show()
```

In [9]:

```
# #Lasso
# class Lasso:
#     def __init__(self, l):
#         self.l = l

#     def __call__(self, theta): #__call__ allows us to call class as method
#         return self.l * np.sum(np.abs(theta))

#     def derivation(self, theta):
#         return self.l * np.sign(theta)
```

In [10]:

```
# #Ridge
# class Ridge:
#     def __init__(self, l):
#         self.l = l

#     def __call__(self, theta): #__call__ allows us to call class as method
#         return self.l * np.sum(np.square(theta))

#     def derivation(self, theta):
#         return self.l * 2 * theta
```

In [11]:

```
# class NoRegularization:
#     def __init__(self, l=0):
#         self.l = l

#     def __call__(self, theta):
#         return 0

#     def derivation(self, theta):
#         return 0
```

In [12]:

```
df = pd.read_csv('Cars.csv')

df = df.drop_duplicates()
df = df[~df['fuel'].isin(['CNG', 'LPG'])]

owner_map = {
    'First Owner': 1, 'Second Owner': 2, 'Third Owner': 3,
    'Fourth & Above Owner': 4, 'Test Drive Car': 5
}
df['owner'] = df['owner'].map(owner_map)
df = df[df['owner'] != 5]

df = df.drop(columns=['torque'])

df['mileage'] = df['mileage'].str.split().str[0].astype(float)
df['engine'] = df['engine'].str.split().str[0].astype(float)
df['max_power'] = df['max_power'].str.split().str[0].astype(float)

df['name'] = df['name'].str.split().str[0]

df.shape
```

Out[12]:

```
(6827, 12)
```

In [13]:

```
X = df.drop(columns=['selling_price'])
y = np.log(df['selling_price'])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

num_features = ['year', 'km_driven', 'owner', 'mileage', 'engine', 'max_power', 'seats']
cat_features = ['name', 'fuel', 'seller_type', 'transmission']

for col in ['mileage', 'engine', 'max_power', 'seats']:
    median = X_train[col].median()
    X_train[col] = X_train[col].fillna(median)
    X_test[col] = X_test[col].fillna(median)

X_train = pd.get_dummies(X_train, columns=cat_features, drop_first=True)
X_test = pd.get_dummies(X_test, columns=cat_features, drop_first=True)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

bool_cols = X_train.select_dtypes(include=bool).columns
X_train[bool_cols] = X_train[bool_cols].astype(int)
X_test[bool_cols] = X_test[bool_cols].astype(int)

scaler = StandardScaler()
X_train[num_features] = scaler.fit_transform(X_train[num_features])
X_test[num_features] = scaler.transform(X_test[num_features])

print(X_train.shape, X_test.shape)
```

```
(5461, 41) (1366, 41)
```

In [14]:

```
feature_names = X_train.columns.tolist()

X_train_np = X_train.to_numpy().astype(float)
X_test_np  = X_test.to_numpy().astype(float)
y_train_np = y_train.to_numpy().astype(float)
y_test_np  = y_test.to_numpy().astype(float)

X_train_np = np.concatenate([np.ones((X_train_np.shape[0], 1)), X_train_np], axis=1)
X_test_np  = np.concatenate([np.ones((X_test_np.shape[0], 1)), X_test_np], axis=1)

print(X_train_np.shape, X_test_np.shape)
```

```
(5461, 42) (1366, 42)
```

In [15]:

```
poly = PolynomialFeatures(degree=2, include_bias=False)
poly_scaler = StandardScaler()

X_train_poly_num = poly_scaler.fit_transform(poly.fit_transform(X_train[num_features]))
X_test_poly_num  = poly_scaler.transform(poly.transform(X_test[num_features]))

cat_cols = [c for c in X_train.columns if c not in num_features]
X_train_poly = np.concatenate([X_train_poly_num, X_train[cat_cols].to_numpy()], axis=1).astype(float)
X_test_poly  = np.concatenate([X_test_poly_num,  X_test[cat_cols].to_numpy()], axis=1).astype(float)

X_train_poly = np.concatenate([np.ones((X_train_poly.shape[0], 1)), X_train_poly], axis=1)
X_test_poly  = np.concatenate([np.ones((X_test_poly.shape[0], 1)), X_test_poly], axis=1)

poly_feature_names = poly.get_feature_names_out(num_features).tolist() + cat_cols

print(X_train_poly.shape, X_test_poly.shape)
```

```
(5461, 70) (1366, 70)
```

In [16]:

```
mlflow.set_experiment("car-price-regularization-sweep")

model_types = ['normal', 'lasso', 'ridge', 'polynomial']
momentum_options = [False, True]
methods = ['sto', 'mini', 'batch']
initialise_methods = ['zeros', 'xavier']
learning_rates = [0.01, 0.001, 0.0001]

SWEEP_EPOCHS = 500
SWEEP_SPLITS = 5

results = []

for model_type in model_types:

    if model_type == 'polynomial':
        Xtr = X_train_poly
    else:
        Xtr = X_train_np
```

```python
    for use_momentum in momentum_options:
        for method in methods:
            for initialise_method in initialise_methods:
                for lr in learning_rates:

                    run_name = (
                        f"{model_type}_{method}_{initialise_method}"
                        f"_mom{use_momentum}_lr{lr}"
                    )

                    try:
                        with mlflow.start_run(run_name=run_name):

                            mlflow.log_params({
                                "model_type": model_type,
                                "method": method,
                                "initialise_method": initialise_method,
                                "use_momentum": use_momentum,
                                "momentum": 0.9,
                                "lr": lr,
                                "epochs": SWEEP_EPOCHS,
                                "cv_splits": SWEEP_SPLITS
                            })

                            if model_type == 'lasso':
                                regularization = Lasso(0.01)
                            elif model_type == 'ridge':
                                regularization = Ridge(0.01)
                            else:
                                regularization = NoRegularization()

                            model = LinearRegression(
                                regularization=regularization,
                                lr=lr,
                                method=method,
                                num_epochs=SWEEP_EPOCHS,
                                cv=KFold(
                                    n_splits=SWEEP_SPLITS,
                                    shuffle=True,
                                    random_state=42
                                ),
                                initialise_method=initialise_method,
                                use_momentum=use_momentum,
                                momentum=0.9
                            )

                            model.fit(Xtr, y_train_np)

                            cv_mse = float(np.mean(model.kfold_scores))
                            cv_r2 = float(np.mean(model.kfold_r2_scores))

                            mlflow.log_metric("cv_mse", cv_mse)
                            mlflow.log_metric("cv_r2", cv_r2)

                            results.append({
                                "model_type": model_type,
                                "method": method,
                                "initialise_method": initialise_method,
                                "use_momentum": use_momentum,
                                "lr": lr,
                                "cv_mse": cv_mse,
                                "cv_r2": cv_r2
                            })

                            print(
                                f"{run_name}: "
                                f"MSE={cv_mse:.4f}, "
                                f"R2={cv_r2:.4f}"
                            )

                    except Exception as e:
                        print(f"Run {run_name} failed: {e}")

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by=["cv_mse", "cv_r2"],
    ascending=[True, False]
)

results_df.head(10)
```

```
2026/09/13 18:09:42 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/09/13 18:09:42 INFO mlflow.store.db.utils: Updating database tables
2026/09/13 18:09:45 INFO mlflow.tracking.fluent: Experiment with name 'car-price-regularization-sweep' does not exist. Creating a new experiment.
Fold 0: MSE=0.08484359960192776, R2=0.866215541158414
Fold 1: MSE=0.08334021155905112, R2=0.8533853816738173
Fold 2: MSE=0.20033138905828152, R2=0.6373732379786116
Fold 3: MSE=0.07532515545274239, R2=0.8739270251410041
Fold 4: MSE=0.06804445586160433, R2=0.8817718421968912
normal_sto_zeros_momFalse_lr0.01: MSE=0.1024, R2=0.8225
Fold 0: MSE=0.07656950298437705, R2=0.8792624361932266
Fold 1: MSE=0.08068394033431925, R2=0.8580583742724615
Fold 2: MSE=0.06451196736982114, R2=0.8832246611531164
Fold 3: MSE=0.07128458427568907, R2=0.8806897968254279
Fold 4: MSE=0.07582611277311593, R2=0.8682511085874541
normal_sto_zeros_momFalse_lr0.001: MSE=0.0738, R2=0.8739
Fold 0: MSE=0.08217231276961194, R2=0.8704277229252035
Fold 1: MSE=0.08654423328901227, R2=0.8477487698359147
Fold 2: MSE=0.06773638282631896, R2=0.8773880354406007
Fold 3: MSE=0.0778820092094436, R2=0.8696475761647726
Fold 4: MSE=0.08219290457520038, R2=0.8571887221469281
normal_sto_zeros_momFalse_lr0.0001: MSE=0.0793, R2=0.8645
Fold 0: MSE=0.07582476551323362, R2=0.8804367651941583
Fold 1: MSE=0.08265266845340058, R2=0.8545949282796155
Fold 2: MSE=0.0682533354730825, R2=0.8764522816114307
Fold 3: MSE=0.07632732606060898, R2=0.8722496754018555
Fold 4: MSE=0.0729783122539911, R2=0.8731992003151867
normal_sto_xavier_momFalse_lr0.01: MSE=0.0752, R2=0.8714
Fold 0: MSE=0.08183312435494211, R2=0.8709625674946802
Fold 1: MSE=0.0825917433033779, R2=0.8547021096444085
Fold 2: MSE=0.07195455010919861, R2=0.8697525852465715
Fold 3: MSE=0.07111607611299314, R2=0.8809718317609208
Fold 4: MSE=0.07388223891519693, R2=0.8716286155762795
normal_sto_xavier_momFalse_lr0.001: MSE=0.0763, R2=0.8696
Fold 0: MSE=0.08185993171935445, R2=0.8709202966721585
Fold 1: MSE=0.09028534095652126, R2=0.8411673001884523
Fold 2: MSE=0.0738909188336802, R2=0.8662474973822132
Fold 3: MSE=0.07518700703455869, R2=0.874158246463387
Fold 4: MSE=0.09498285816281511, R2=0.8349660056610108
normal_sto_xavier_momFalse_lr0.0001: MSE=0.0832, R2=0.8575
Fold 0: MSE=0.07738604197886609, R2=0.8779748879513639
Fold 1: MSE=0.08318731381393411, R2=0.853654364006882
Fold 2: MSE=0.06438587518428422, R2=0.8834529049703853
Fold 3: MSE=0.07610930582598428, R2=0.8726145795217791
Fold 4: MSE=0.0787656729142472, R2=0.8631435832815827
normal_mini_zeros_momFalse_lr0.01: MSE=0.0760, R2=0.8702
Fold 0: MSE=0.35349301365140384, R2=0.44259942108161243
Fold 1: MSE=0.39054461753618036, R2=0.31294210839817027
Fold 2: MSE=0.36682248538629136, R2=0.33600195786806974
Fold 3: MSE=0.32958763678394054, R2=0.4483636496154779
Fold 4: MSE=0.3519869324410117, R2=0.3884179678369617
normal_mini_zeros_momFalse_lr0.001: MSE=0.3585, R2=0.3857
Fold 0: MSE=2.3941345425020444, R2=-2.7751580044383966
Fold 1: MSE=2.6301875866006563, R2=-3.627105474318159
Fold 2: MSE=2.4692235901271378, R2=-3.4696268488116724
Fold 3: MSE=2.317125030612829, R2=-2.8782109903892095
Fold 4: MSE=2.2893708259860346, R2=-2.9778126205465227
normal_mini_zeros_momFalse_lr0.0001: MSE=2.4200, R2=-3.1456
Fold 0: MSE=0.07778771445663171, R2=0.8773415162495318
Fold 1: MSE=0.08350587306942285, R2=0.8530939449392551
Fold 2: MSE=0.06602808197949916, R2=0.8804802897675572
Fold 3: MSE=0.07232914145974903, R2=0.8789415039634584
Fold 4: MSE=0.08716713979085824, R2=0.8485459215163712
```

```
normal_mini_xavier_momFalse_lr0.01: MSE=0.0774, R2=0.8677
Fold 0: MSE=0.34160693165137557, R2=0.4613418254065271З
Fold 1: MSE=0.38477739824799, R2=0.3230879748283545
Fold 2: MSE=0.361125396866585l, R2=0.3463144544396842
Fold 3: MSE=0.32753615594331, R2=0.4517972474738559７
Fold 4: MSE=0.35307628029027166, R2=0.38652521128840855
normal_mini_xavier_momFalse_lr0.001: MSE=0.3536, R2=0.3938
Fold 0: MSE=2.3948233873817912, R2=-2.776244199978132
Fold 1: MSE=2.679169113006227, R2=-3.7132752555635324
Fold 2: MSE=2.4787796723453206, R2=-3.4869246430748655
Fold 3: MSE=2.301362186484637, R2=-2.851828454043478６
Fold 4: MSE=2.1909126165431116, R2=-2.806740156587004
normal_mini_xavier_momFalse_lr0.0001: MSE=2.4090, R2=-3.1270
Fold 0: MSE=2.309993836267945, R2=-2.64248189330105
Fold 1: MSE=2.537504496726946, R2=-3.4640545821627313
Fold 2: MSE=2.386133293985976, R2=-3.319222235801667
Fold 3: MSE=2.2335674363764544, R2=-2.738359244792087
Fold 4: MSE=2.206378598148004, R2=-2.833612507766976
normal_batch_zeros_momFalse_lr0.01: MSE=2.3347, R2=-2.9995
Fold 0: MSE=12.367179375344739, R2=-18.501016080059447
Fold 1: MSE=12.178680850061042, R2=-20.42510333421597
Fold 2: MSE=11.9468759052844, R2=-20.62545243743237
Fold 3: MSE=11.988369710117l5, R2=-19.065135265630847
Fold 4: MSE=11.709128143165664, R2=-19.344767730417903
normal_batch_zeros_momFalse_lr0.001: MSE=12.0380, R2=-19.5923
Fold 0: MSE=124.44202292473787, R2=-195.22468603701265
Fold 1: MSE=123.6115123402646５, R2=-216.46110747089728
Fold 2: MSE=124.03590114438737, R2=-223.52166591481458
Fold 3: MSE=124.27203366916319, R2=-206.9961851028386
Fold 4: MSE=123.43812485812303, R2=-213.475403173603
normal_batch_zeros_momFalse_lr0.0001: MSE=123.9599, R2=-211.1358
Fold 0: MSE=2.332559383413041, R2=-2.678064064819412
Fold 1: MSE=2.6115840376110184, R2=-3.594377548823326
Fold 2: MSE=2.3889833604639725, R2=-3.324381239507076
Fold 3: MSE=2.3001815280099267, R2=-2.849852366170781
Fold 4: MSE=2.238005955708373, R2=-2.8885654671696925
normal_batch_xavier_momFalse_lr0.01: MSE=2.3743, R2=-3.0670
Fold 0: MSE=12.106352583019916, R2=-18.089734953064802
Fold 1: MSE=12.00299177310081, R2=-20.116025801525133
Fold 2: MSE=11.983056966429457, R2=-20.690945025045348
Fold 3: MSE=12.037139318096743, R2=-19.146761775708903
Fold 4: MSE=11.673895966950711, R2=-19.283551350089127
normal_batch_xavier_momFalse_lr0.001: MSE=11.9607, R2=-19.4654
Fold 0: MSE=125.0204045775006, R2=-196.13669916212532
Fold 1: MSE=126.37634930980697, R2=-221.32509220816115
Fold 2: MSE=126.60469004172779, R2=-228.17151936283338
Fold 3: MSE=129.88821079028892, R2=-216.3960748573253
Fold 4: MSE=120.29040365429925, R2=-208.0061952198678
normal_batch_xavier_momFalse_lr0.0001: MSE=125.6360, R2=-214.0071
Fold 0: MSE=0.07505118962171481, R2=0.88165656766017
Fold 1: MSE=0.08294768886830338, R2=0.854075919451581
Fold 2: MSE=0.0634639086371227, R2=0.8851217884402242
Fold 3: MSE=0.071440592524973, R2=0.8804286832044662
Fold 4: MSE=0.07296811399039449, R2=0.8732169199354343
normal_sto_zeros_momTrue_lr0.01: MSE=0.0732, R2=0.8749
Fold 0: MSE=0.08519899667475378, R2=0.86565513819007
Fold 1: MSE=0.09225593060242515, R2=0.8377005793413734
Fold 2: MSE=0.07320378350323843, R2=0.8674913047611761
Fold 3: MSE=0.07998237240459367, R2=0.8661321630905434
Fold 4: MSE=0.08402308148261661, R2=0.8540087651397371
normal_sto_zeros_momTrue_lr0.001: MSE=0.0829, R2=0.8582
Fold 0: MSE=0.9475335126449651, R2=-0.4941051395536664
Fold 1: MSE=1.0439279461597852, R2=-0.8365095855054892
Fold 2: MSE=0.9989960257085077, R2=-0.8083171877249871
Fold 3: MSE=0.8999175590972815, R2=-0.5062070980314173
Fold 4: MSE=0.920975146363261２, R2=-0.6002067113070695
normal_sto_zeros_momTrue_lr0.0001: MSE=0.9623, R2=-0.6491
Fold 0: MSE=0.07564503902100275, R2=0.880720164432467６
Fold 1: MSE=0.08431522983747901, R2=0.8516701000579137
Fold 2: MSE=0.06852219147489473, R2=0.8759656160826512
Fold 3: MSE=0.07216479740547５2, R2=0.8792165693609097
Fold 4: MSE=0.07634329851279743, R2=0.8673524914044621
normal_sto_xavier_momTrue_lr0.01: MSE=0.0754, R2=0.8710
Fold 0: MSE=0.08042334907760004, R2=0.8731855521800584
Fold 1: MSE=0.09140431812444044, R2=0.8391987617443912
Fold 2: MSE=0.0758116033832427, R2=0.8627707999842346
Fold 3: MSE=0.08186658359191883, R2=0.8629785272537382
Fold 4: MSE=0.08519562502714474, R2=0.8519714549510063
normal_sto_xavier_momTrue_lr0.001: MSE=0.0829, R2=0.8580
Fold 0: MSE=0.9175665744152068, R2=-0.446852186673101
Fold 1: MSE=1.0343113025083797, R2=-0.8195916954239322
Fold 2: MSE=0.9967026277391486, R2=-0.8041658289011677
Fold 3: MSE=0.9145494701402208, R2=-0.5306967727220042
Fold 4: MSE=0.9273597713186689, R2=-0.6113000830914934
normal_sto_xavier_momTrue_lr0.0001: MSE=0.9581, R2=-0.6425
Fold 0: MSE=0.353862892821560２, R2=0.4420161822180055４
Fold 1: MSE=0.3907697148683427, R2=0.31254611037004143
Fold 2: MSE=0.3664224555768983, R2=0.33672606563357843
Fold 3: MSE=0.32924636186196726, R2=0.4489348471709206
Fold 4: MSE=0.5613399706457298, R2=0.02466424647912413
normal_mini_zeros_momTrue_lr0.01: MSE=0.4003, R2=0.3130
Fold 0: MSE=2.393619064037901, R2=-2.774345179337935
Fold 1: MSE=2.629629977121569, R2=-3.626124510874197
Fold 2: MSE=2.4692100052461883, R2=-3.4696022583497665
Fold 3: MSE=2.3166244560971343, R2=-2.877373170434469
Fold 4: MSE=2.2889170510946477, R2=-2.9770241805658273
normal_mini_zeros_momTrue_lr0.001: MSE=2.4196, R2=-3.1449
Fold 0: MSE=15.982327898174615, R2=-24.201513124361707
Fold 1: MSE=15.706212396474287, R2=-26.63084341617498
Fold 2: MSE=15.50801980778786, R2=-27.07160193266374
Fold 3: MSE=15.572641056261288, R2=-25.064190277123657
Fold 4: MSE=15.23626032423846, R2=-25.473207363243354
normal_mini_zeros_momTrue_lr0.0001: MSE=15.6011, R2=-25.6883
Fold 0: MSE=0.3558277927040922, R2=0.43891785696193686
Fold 1: MSE=0.38509340339455417, R2=0.32253204902632093
Fold 2: MSE=0.3625914866557739, R2=0.343660634708361６
Fold 3: MSE=0.3320160972509754, R2=0.44429909463956907
Fold 4: MSE=0.3515873387766623, R2=0.3891122672065077
normal_mini_xavier_momTrue_lr0.01: MSE=0.3574, R2=0.3877
Fold 0: MSE=2.4469396612677463, R2=-2.8584230270362183
Fold 1: MSE=2.622524529554411, R2=-3.61362439282065２
Fold 2: MSE=2.4467663175308076, R2=-3.428976164544367
Fold 3: MSE=2.2707434423203754, R2=-2.800581348876111
Fold 4: MSE=2.2556501128012747, R2=-2.9192224275740086
normal_mini_xavier_momTrue_lr0.001: MSE=2.4085, R2=-3.1242
Fold 0: MSE=16.0619933664774, R2=-24.327132518343756
Fold 1: MSE=15.924729315208515, R2=-27.015264982171747
Fold 2: MSE=14.893019064975425, R2=-25.958367860583593
Fold 3: MSE=15.507025244430817, R2=-24.954367993378813
Fold 4: MSE=15.70168984574766, R2=-26.281897420624876
normal_mini_xavier_momTrue_lr0.0001: MSE=15.6177, R2=-25.7074
Fold 0: MSE=12.55505061132312, R2=-18.797258245116012
Fold 1: MSE=12.374205078776457, R2=-20.769075465200064
Fold 2: MSE=12.137396391118706, R2=-20.97032014489251
Fold 3: MSE=12.17907525874388, R2=-19.384322337904102
Fold 4: MSE=11.90267910055749, R2=-19.68106512369864
```

```
normal_batch_zeros_momTrue_lr0.01: MSE=12.2297, R2=-19.9204
Fold 0: MSE=123.89183432186715, R2=-194.3571287334421
Fold 1: MSE=123.06189858889445, R2=-215.4942103527292
Fold 2: MSE=123.48372706445869, R2=-222.5221565537607
Fold 3: MSE=123.71948671816982, R2=-206.07137801226835
Fold 4: MSE=122.88668655435029, R2=-212.51727169953077
normal_batch_zeros_momTrue_lr0.001: MSE=123.4087, R2=-210.1924
Fold 0: MSE=161.82371606087182, R2=-254.16949283757506
Fold 1: MSE=160.99876735234898, R2=-282.23389615618214
Fold 2: MSE=161.5560601753219, R2=-291.4382008316521
Fold 3: MSE=161.81910202838662, R2=-269.8393425690212
Fold 4: MSE=160.93799241664578, R2=-278.6319277304617
normal_batch_zeros_momTrue_lr0.0001: MSE=161.4271, R2=-275.2626
Fold 0: MSE=12.671378646434697, R2=-18.980688501476255
Fold 1: MSE=12.02261066427614, R2=-20.15053994767193
Fold 2: MSE=12.37311487290604, R2=-21.397002304892265
Fold 3: MSE=11.981961748924867, R2=-19.05441015360902
Fold 4: MSE=11.844445123412608, R2=-19.579882804695444
normal_batch_xavier_momTrue_lr0.01: MSE=12.1787, R2=-19.8325
Fold 0: MSE=123.7057490760131, R2=-194.06370278225907
Fold 1: MSE=120.08058044900945, R2=-210.24938539954974
Fold 2: MSE=124.99123630288446, R2=-225.25095105979105
Fold 3: MSE=124.50581198766763, R2=-207.38746379180034
Fold 4: MSE=120.7067969484778, R2=-208.7296841723187
normal_batch_xavier_momTrue_lr0.001: MSE=122.7980, R2=-209.1362
Fold 0: MSE=166.51239153912965, R2=-261.56276604245556
Fold 1: MSE=155.30378099645378, R2=-272.21510408303305
Fold 2: MSE=171.2938417570752, R2=-309.0648953844234
Fold 3: MSE=158.28359753064998, R2=-263.9219094488644
Fold 4: MSE=153.37708438761075, R2=-265.49474827517037
normal_batch_xavier_momTrue_lr0.0001: MSE=160.9541, R2=-274.4519
Fold 0: MSE=0.08055410925449179, R2=0.8729793648001551
Fold 1: MSE=0.09588218492550714, R2=0.8313211631677582
Fold 2: MSE=0.09273326817726427, R2=0.8321403104682747
Fold 3: MSE=0.08612091555438015, R2=0.8558579805609179
Fold 4: MSE=0.08383241992641034, R2=0.8543400421595724
lasso_sto_zeros_momFalse_lr0.01: MSE=0.0878, R2=0.8493
Fold 0: MSE=0.08044273929702113, R2=0.8731549769802291
Fold 1: MSE=0.09287067808439245, R2=0.8366190969963022
Fold 2: MSE=0.07943036989691608, R2=0.8562203458116084
Fold 3: MSE=0.08976051621l793, R2=0.8497663199540351
Fold 4: MSE=0.0779313714343535, R2=0.8645931933309431
lasso_sto_zeros_momFalse_lr0.001: MSE=0.0841, R2=0.8561
Fold 0: MSE=0.08019413744444184, R2=0.8735469813797405
Fold 1: MSE=0.09282561831080441, R2=0.836698367511494
Fold 2: MSE=0.0787522230619532, R2=0.857447882804656
Fold 3: MSE=0.0882347152944386, R2=0.8523200785163276
Fold 4: MSE=0.07810111055832275, R2=0.8642982693186941
lasso_sto_zeros_momFalse_lr0.0001: MSE=0.0836, R2=0.8569
Fold 0: MSE=0.09429783264572911, R2=0.8513077642906058
Fold 1: MSE=0.09313450792769877, R2=0.8361549595642536
Fold 2: MSE=0.08304733363640968, R2=0.8496731549027502
Fold 3: MSE=0.0867408867310938, R2=0.8548203244139729
Fold 4: MSE=0.07974568881382886, R2=0.8614407924669871
lasso_sto_xavier_momFalse_lr0.01: MSE=0.0874, R2=0.8507
Fold 0: MSE=0.07974316142297348, R2=0.8742580966938684
Fold 1: MSE=0.09277430634469433, R2=0.8367886370726825
Fold 2: MSE=0.08009835520205638, R2=0.8550112025544305
Fold 3: MSE=0.0868271108061206, R2=0.8546760097347963
Fold 4: MSE=0.07889537041961663, R2=0.8629182321205319
lasso_sto_xavier_momFalse_lr0.001: MSE=0.0837, R2=0.8567
Fold 0: MSE=0.08036933375839762, R2=0.8732707254905092
Fold 1: MSE=0.09308152523074281, R2=0.8362481683256278
Fold 2: MSE=0.07858947065716942, R2=0.8577424865501567
Fold 3: MSE=0.08630957840629919, R2=0.8555422123843526
Fold 4: MSE=0.07827832097214257, R2=0.8639903638397811
lasso_sto_xavier_momFalse_lr0.0001: MSE=0.0833, R2=0.8574
Fold 0: MSE=0.08003440894138199, R2=0.87379884706488
Fold 1: MSE=0.09302065431740703, R2=0.8363552542756025
Fold 2: MSE=0.07886879484706713, R2=0.8572368721928012
Fold 3: MSE=0.0863876696896799, R2=0.8554115096948302
Fold 4: MSE=0.07823344181230125, R2=0.8640683419839901
lasso_mini_zeros_momFalse_lr0.01: MSE=0.0833, R2=0.8574
Fold 0: MSE=0.322448732081 2539, R2=0.49155116793670417
Fold 1: MSE=0.366373572027245, R2=0.35546454199346234
Fold 2: MSE=0.34691006552188336, R2=0.37204611636656615
Fold 3: MSE=0.3093412517251727, R2=0.48225036354467776
Fold 4: MSE=0.32024217967427693, R2=0.44357490299064517
lasso_mini_zeros_momFalse_lr0.001: MSE=0.3331, R2=0.4290
Fold 0: MSE=2.386958639725199, R2=-2.763842781201707
Fold 1: MSE=2.6262123582927193, R2=-3.6201121325660246
Fold 2: MSE=2.465768826077444, R2=-3.4633732611598447
Fold 3: MSE=2.312686625745099, R2=-2.8707823577904437
Fold 4: MSE=2.280038697600632, R2=-2.9615979218849264
lasso_mini_zeros_momFalse_lr0.0001: MSE=2.4143, R2=-3.1359
Fold 0: MSE=0.08003293600699361, R2=0.8738011696411377
Fold 1: MSE=0.09276530605302005, R2=0.8368044706577427
Fold 2: MSE=0.07857971905872144, R2=0.857760138255096
Fold 3: MSE=0.08584871244156926, R2=0.8563135714719712
Fold 4: MSE=0.07797901736254267, R2=0.8645104078894946
lasso_mini_xavier_momFalse_lr0.01: MSE=0.0830, R2=0.8578
Fold 0: MSE=0.32399686294466523, R2=0.4891100191553087
Fold 1: MSE=0.3646064770600072, R2=0.3585732688531743
Fold 2: MSE=0.3420950164023342, R2=0.3807620029752 6963
Fold 3: MSE=0.30539025925352276, R2=0.488863205848852
Fold 4: MSE=0.31664898953553033, R2=0.4498181192139381
lasso_mini_xavier_momFalse_lr0.001: MSE=0.3305, R2=0.4334
Fold 0: MSE=2.3239935614723657, R2=-2.66455716673584
Fold 1: MSE=2.6267939071982296, R2=-3.6211352109722865
Fold 2: MSE=2.436124060222385, R2=-3.4097122472596473
Fold 3: MSE=2.3134930814727097, R2=-2.8721321362550327
Fold 4: MSE=2.218615868691098, R2=-2.85487493002502
lasso_mini_xavier_momFalse_lr0.0001: MSE=2.3838, R2=-3.0845
Fold 0: MSE=2.3015126045207683, R2=-2.6291083800964037
Fold 1: MSE=2.532130843854507, R2=-3.4546010896628747
Fold 2: MSE=2.381317221588116, R2=-3.3105044969215722
Fold 3: MSE=2.2279200458656097, R2=-2.7289071126642037
Fold 4: MSE=2.1959179264978896, R2=-2.815436949994986
lasso_batch_zeros_momFalse_lr0.01: MSE=2.3278, R2=-2.9877
Fold 0: MSE=12.434438868376636, R2=-18.60707328318909
Fold 1: MSE=12.234578173177757, R2=-20.52343959399863
Fold 2: MSE=12.01201318605017, R2=-20.743359677640726
Fold 3: MSE=12.051483253244601, R2=-19.170769460318457
Fold 4: MSE=11.761290145905013, R2=-19.435399912174738
lasso_batch_zeros_momFalse_lr0.001: MSE=12.0988, R2=-19.6960
Fold 0: MSE=124.47890828149805, R2=-195.28284820265029
Fold 1: MSE=123.64530760787933, R2=-216.52056112681996
Fold 2: MSE=124.07258268772517, R2=-223.5880644425194
Fold 3: MSE=124.3079872212982, R2=-207.05636116549874
Fold 4: MSE=123.47112338753709, R2=-213.53273856255478
lasso_batch_zeros_momFalse_lr0.0001: MSE=123.9952, R2=-211.1961
Fold 0: MSE=2.3192633803699056, R2=-2.657098445960384
Fold 1: MSE=2.5409689806958573, R2=-3.4701494070413466
Fold 2: MSE=2.40065361898406, R2=-3.345505977267771
Fold 3: MSE=2.2322853940669645, R2=-2.7362134690962914
Fold 4: MSE=2.2155851225981893, R2=-2.849608967900753
lasso_batch_xavier_momFalse_lr0.01: MSE=2.3418, R2=-3.0117
Fold 0: MSE=12.399078290582366, R2=-18.551315444215643
Fold 1: MSE=12.172284558609796, R2=-20.41385078502892
Fold 2: MSE=11.972607787366925, R2=-20.67203060535785
Fold 3: MSE=12.05548811296119, R2=-19.17747246113310 3
Fold 4: MSE=12.012203379710453, R2=-19.871365032706723
lasso_batch_xavier_momFalse_lr0.001: MSE=12.1223, R2=-19.7372
Fold 0: MSE=122.63512036079786, R2=-192.37549667175267
Fold 1: MSE=123.52959569788541, R2=-216.3169972384858
Fold 2: MSE=120.5230315082291 8, R2=-217.1628993353411
Fold 3: MSE=117.04761076464622, R2=-194.90454743229515
Fold 4: MSE=124.94819485041991, R2=-216.0991701077693
lasso_batch_xavier_momFalse_lr0.0001: MSE=121.7367, R2=-207.3718
Fold 0: MSE=0.07919086377340857, R2=0.8751289796186053
Fold 1: MSE=0.09360179717316655, R2=0.8353328902043307
Fold 2: MSE=0.07793996103603568, R2=0.8589181863340032
Fold 3: MSE=0.08561856681040407, R2=0.8566987700712625
Fold 4: MSE=0.07886170750942746, R2=0.862976721880021 6
```

```
lasso_sto_zeros_momTrue_lr0.01: MSE=0.0830, R2=0.8578
Fold 0: MSE=0.08038953708287745, R2=0.8732388681571921
Fold 1: MSE=0.0942365589416303, R2=0.8342161981217167
Fold 2: MSE=0.0787550737387629, R2=0.8574427226961681
Fold 3: MSE=0.0863556251887671, R2=0.8554651431129581
Fold 4: MSE=0.07821457653155713, R2=0.8641011206631722
lasso_sto_zeros_momTrue_lr0.001: MSE=0.0836, R2=0.8569
Fold 0: MSE=0.920201864748305, R2=-0.4510076078569989
Fold 1: MSE=1.0215387065608383, R2=-0.7971217587048918
Fold 2: MSE=0.9809766097640483, R2=-0.7756996209613738
Fold 3: MSE=0.8821186782924778, R2=-0.476416812983415
Fold 4: MSE=0.8912757773462935, R2=-0.5486036579454194
lasso_sto_zeros_momTrue_lr0.0001: MSE=0.9392, R2=-0.6098
Fold 0: MSE=0.08036718242779742, R2=0.8732741177864689
Fold 1: MSE=0.0928884392261808, R2=0.8365878510590095
Fold 2: MSE=0.0781132020347245, R2=0.858604596822648
Fold 3: MSE=0.08789884520431523, R2=0.8528822299141375
Fold 4: MSE=0.08094212575324551, R2=0.8593619671830798
lasso_sto_xavier_momTrue_lr0.01: MSE=0.0840, R2=0.8561
Fold 0: MSE=0.08024959532979455, R2=0.873459533378769
Fold 1: MSE=0.09346809961091578, R2=0.8355680949955574
Fold 2: MSE=0.07956405111911555, R2=0.8559783648171316
Fold 3: MSE=0.08608847693036481, R2=0.8559122736294866
Fold 4: MSE=0.078146587375466, R2=0.8642192527368799
lasso_sto_xavier_momTrue_lr0.001: MSE=0.0835, R2=0.8570
Fold 0: MSE=0.9480422216003895, R2=-0.49490729024775093
Fold 1: MSE=1.0329640188901659, R2=-0.8172215133741596
Fold 2: MSE=0.9887699100360633, R2=-0.7898065427791976
Fold 3: MSE=0.8693707379932918, R2=-0.4550803716951979
Fold 4: MSE=0.8915486799788483, R2=-0.5490778299422123
lasso_sto_xavier_momTrue_lr0.0001: MSE=0.9461, R2=-0.6212
Fold 0: MSE=0.32237589402868555, R2=0.49166602161443174
Fold 1: MSE=0.36581407618838446, R2=0.35644882397853395
Fold 2: MSE=0.34706281925995025, R2=0.3717696115528061
Fold 3: MSE=0.30951077592701287, R2=0.48196662804744406
Fold 4: MSE=0.32041892012914397, R2=0.44326781407173743
lasso_mini_zeros_momTrue_lr0.01: MSE=0.3330, R2=0.4290
Fold 0: MSE=2.386982210171156, R2=-2.7638799479340457
Fold 1: MSE=2.625806460167468, R2=-3.619398064319739
Fold 2: MSE=2.4660849297531615, R2=-3.463945451333972
Fold 3: MSE=2.3125372149940726, R2=-2.8705322865128657
Fold 4: MSE=2.27995633312387, R2=-2.96145481249809
lasso_mini_zeros_momTrue_lr0.001: MSE=2.4143, R2=-3.1358
Fold 0: MSE=16.055712357734357, R2=-24.317228396411053
Fold 1: MSE=15.769300635398961, R2=-26.74183015232285
Fold 2: MSE=15.57826491523853, R2=-27.198755026257423
Fold 3: MSE=15.642943683217299, R2=-25.181856961878225
Fold 4: MSE=15.294865422952292, R2=-25.575034510967157
lasso_mini_zeros_momTrue_lr0.0001: MSE=15.6682, R2=-25.8029
Fold 0: MSE=0.3231601121715544, R2=0.4904294380612856
Fold 1: MSE=0.36826010772957063, R2=0.3521456913836436
Fold 2: MSE=0.33977739372149435, R2=0.3849572117854866
Fold 3: MSE=0.301098902218261, R2=0.49604572202642516
Fold 4: MSE=0.32527987296251376, R2=0.4348218431049623
lasso_mini_xavier_momTrue_lr0.01: MSE=0.3315, R2=0.4317
Fold 0: MSE=2.373835145032364, R2=-2.743149179753526
Fold 1: MSE=2.6572523675319992, R2=-3.674718654703704
Fold 2: MSE=2.426491325715301, R2=-3.392275702042565
Fold 3: MSE=2.268906610457105, R2=-2.797507012607883
Fold 4: MSE=2.2514780771521905, R2=-2.91197345948639
lasso_mini_xavier_momTrue_lr0.001: MSE=2.3956, R2=-3.1039
Fold 0: MSE=15.659487085082612, R2=-23.692446044768943
Fold 1: MSE=16.016172501762675, R2=-27.176134598943342
Fold 2: MSE=15.709465791217504, R2=-27.436246260428337
Fold 3: MSE=16.096628512005154, R2=-25.941196861940742
Fold 4: MSE=15.978875602234876, R2=-26.76351140289255
lasso_mini_xavier_momTrue_lr0.0001: MSE=15.8921, R2=-26.2019
Fold 0: MSE=12.622736847850147, R2=-18.903988352835135
Fold 1: MSE=12.430399364064847, R2=-20.86793414980794
Fold 2: MSE=12.203074616854016, R2=-21.089206568261627
Fold 3: MSE=12.242611863411545, R2=-19.49066461778055
Fold 4: MSE=11.955074520616169, R2=-19.772102871189144
lasso_batch_zeros_momTrue_lr0.01: MSE=12.2908, R2=-20.0248
Fold 0: MSE=123.92917658243339, R2=-194.41601136153915
Fold 1: MSE=123.09610504213639, R2=-215.5543873788315
Fold 2: MSE=123.52086164967213, R2=-222.58937514820192
Fold 3: MSE=123.75588139353, R2=-206.1322923902621
Fold 4: MSE=122.92008329447042, R2=-212.57529898494346
lasso_batch_zeros_momTrue_lr0.001: MSE=123.4444, R2=-210.2535
Fold 0: MSE=161.82823021091306, R2=-254.17661091275377
Fold 1: MSE=161.0029244672734, R2=-282.24120947836576
Fold 2: MSE=161.5605588331224, R2=-291.4463440074136
Fold 3: MSE=161.82351506891325, R2=-269.8467287488973
Fold 4: MSE=160.94206064661853, R2=-278.6389963347897
lasso_batch_zeros_momTrue_lr0.0001: MSE=161.4315, R2=-275.2700
Fold 0: MSE=12.700393333713988, R2=-19.026439910590558
Fold 1: MSE=12.439393587232178, R2=-20.883757058967277
Fold 2: MSE=12.184032707404652, R2=-21.054738150712l5
Fold 3: MSE=12.061730942492531, R2=-19.187921189527
Fold 4: MSE=12.05615877078494, R2=-19.94773811625113
lasso_batch_xavier_momTrue_lr0.01: MSE=12.2883, R2=-20.0201
Fold 0: MSE=119.3852353921604, R2=-187.250960420739
Fold 1: MSE=125.28115524514882, R2=-219.39839371797095
Fold 2: MSE=123.70092684875853, R2=-222.91531738024247
Fold 3: MSE=124.53543014416155, R2=-207.43703619661764
Fold 4: MSE=127.65481328016432, R2=-220.80195605517213
lasso_batch_xavier_momTrue_lr0.001: MSE=124.1115, R2=-211.5607
Fold 0: MSE=166.18526573672906, R2=-261.046942236622
Fold 1: MSE=162.7432254640823, R2=-285.3027995757106
Fold 2: MSE=162.13701078348217, R2=-292.4897995797156
Fold 3: MSE=158.36841463016992, R2=-264.06386924955945
Fold 4: MSE=163.53227827965478, R2=-283.13953433138795
lasso_batch_xavier_momTrue_lr0.0001: MSE=162.5932, R2=-277.2086
Fold 0: MSE=0.0745161705195736, R2=0.8825002051459168
Fold 1: MSE=0.0849472953991447, R2=0.8505581512237657
Fold 2: MSE=0.06843543501133556, R2=0.8761226569518469
Fold 3: MSE=0.08450361382735301, R2=0.858564885561583
Fold 4: MSE=0.08186653326803668, R2=0.8577557966854363
ridge_sto_zeros_momFalse_lr0.01: MSE=0.0789, R2=0.8651
Fold 0: MSE=0.07342591139711113, R2=0.8842193651931105
Fold 1: MSE=0.08410287266323169, R2=0.8520436852152904
Fold 2: MSE=0.06736172241035182, R2=0.878066221191413
Fold 3: MSE=0.0768540610168706, R2=0.8713680702793836
Fold 4: MSE=0.07718847819225842, R2=0.8658839803369546
ridge_sto_zeros_momFalse_lr0.001: MSE=0.0758, R2=0.8703
Fold 0: MSE=0.07393044091432849, R2=0.8834238047884693
Fold 1: MSE=0.08409434525094572, R2=0.8520586868966424
Fold 2: MSE=0.0680743992823957, R2=0.876776180186398
Fold 3: MSE=0.0789376005059337, R2=0.8678808153239368
Fold 4: MSE=0.08275139183107495, R2=0.856218343023714
ridge_sto_zeros_momFalse_lr0.0001: MSE=0.0776, R2=0.8673
Fold 0: MSE=0.07705981812431202, R2=0.8784892895331863
Fold 1: MSE=0.08846974870320444, R2=0.8443613449391956
Fold 2: MSE=0.11439214752634022, R2=0.7929348253753601
Fold 3: MSE=0.07812464463007057, R2=0.8692414731955753
Fold 4: MSE=0.07795240700152756, R2=0.8645566437498811
ridge_sto_xavier_momFalse_lr0.01: MSE=0.0872, R2=0.8499
Fold 0: MSE=0.07208915731295251, R2=0.8863272074180104
Fold 1: MSE=0.08370381635138087, R2=0.8527457171367141
Fold 2: MSE=0.06775342000920397, R2=0.877357195847801
Fold 3: MSE=0.07760034268778947, R2=0.8701190061417794
Fold 4: MSE=0.0749717807929737, R2=0.8697355219005671
ridge_sto_xavier_momFalse_lr0.001: MSE=0.0752, R2=0.8713
Fold 0: MSE=0.07322936512513549, R2=0.8845292864689283
Fold 1: MSE=0.08382219816585194, R2=0.8525374562717517
Fold 2: MSE=0.06918301033776299, R2=0.8747694450499885
Fold 3: MSE=0.07756258430203972, R2=0.8701822029847031
Fold 4: MSE=0.07987309218398693, R2=0.8612194273967405
ridge_sto_xavier_momFalse_lr0.0001: MSE=0.0767, R2=0.8686
Fold 0: MSE=0.07977979088498749, R2=0.874200338032329
Fold 1: MSE=0.0837265203584123, R2=0.8527057755615355
Fold 2: MSE=0.0676202523909891, R2=0.8775982471499777
Fold 3: MSE=0.07591451029927328, R2=0.8729406120589053
Fold 4: MSE=0.07593030054682408, R2=0.8680700809284752
```

```
ridge_mini_zeros_momFalse_lr0.01: MSE=0.0766, R2=0.8691
Fold 0: MSE=0.24173900078306648, R2=0.6188171936079261
Fold 1: MSE=0.2742857475074107, R2=0.5174682253522154
Fold 2: MSE=0.24689188419769562, R2=0.5530924786333733
Fold 3: MSE=0.22954481332710752, R2=0.6158069995918247
Fold 4: MSE=0.24152287306573178, R2=0.5803507576289066
ridge_mini_zeros_momFalse_lr0.001: MSE=0.2468, R2=0.5771
Fold 0: MSE=2.2855601763670266, R2=-2.603954013971312
Fold 1: MSE=2.5147748092643347, R2=-3.4240678291936337
Fold 2: MSE=2.3530663401222345, R2=-3.259366601265879
Fold 3: MSE=2.2099539078552826, R2=-2.69883688642869
Fold 4: MSE=2.184182707140506, R2=-2.7950468484287287
ridge_mini_zeros_momFalse_lr0.0001: MSE=2.3095, R2=-2.9563
Fold 0: MSE=0.07238662600563599, R2=0.8858581480162442
Fold 1: MSE=0.08626640613138176, R2=0.8482375317662559
Fold 2: MSE=0.08075590372442129, R2=0.853820951277993
Fold 3: MSE=0.07603047037677568, R2=0.8727465277341171
Fold 4: MSE=0.0763201392184212, R2=0.867392730990119
ridge_mini_xavier_momFalse_lr0.01: MSE=0.0784, R2=0.8656
Fold 0: MSE=0.24510443782247093, R2=0.6135104506692175
Fold 1: MSE=0.27835842337695005, R2=0.5103034509053006
Fold 2: MSE=0.24784568116494138, R2=0.5513659778214735
Fold 3: MSE=0.22654512507070929, R2=0.6208276280905071
Fold 4: MSE=0.24230932852440773, R2=0.5789842806853247
ridge_mini_xavier_momFalse_lr0.001: MSE=0.2480, R2=0.5750
Fold 0: MSE=2.2646002298808665, R2=-2.570903611688099
Fold 1: MSE=2.492608519248139, R2=-3.3850722220355056
Fold 2: MSE=2.37334395235717, R2=-3.2960718070793362
Fold 3: MSE=2.258480808445795, R2=-2.7800571730827404
Fold 4: MSE=2.2744431117750543, R2=-2.951875516207485
ridge_mini_xavier_momFalse_lr0.0001: MSE=2.3327, R2=-2.9968
Fold 0: MSE=2.1890514617146173, R2=-2.4517755794889506
Fold 1: MSE=2.409071971670148, R2=-3.238112203452447
Fold 2: MSE=2.257132739024648, R2=-3.0857138786516085
Fold 3: MSE=2.1148017368090866, R2=-2.539579103341709
Fold 4: MSE=2.0899245256574748, R2=-2.631271989573576
ridge_batch_zeros_momFalse_lr0.01: MSE=2.2120, R2=-2.7893
Fold 0: MSE=12.49916374307559, R2=-18.709133808388593
Fold 1: MSE=12.307864031709084, R2=-20.652366290682902
Fold 2: MSE=12.07754132191738, R2=-20.861974418159054
Fold 3: MSE=12.120171033979108, R2=-19.285733349893096
Fold 4: MSE=11.83926638240149, R2=-19.57088467249303
ridge_batch_zeros_momFalse_lr0.001: MSE=12.1688, R2=-19.8160
Fold 0: MSE=124.45455451633276, R2=-195.24444630423153
Fold 1: MSE=123.62405751711366, R2=-216.48317732506663
Fold 2: MSE=124.04850724828974, R2=-223.54448465860358
Fold 3: MSE=124.28465113133431, R2=-207.01730316069754
Fold 4: MSE=123.45073111028137, R2=-213.4973067063833
ridge_batch_zeros_momFalse_lr0.0001: MSE=123.9725, R2=-211.1573
Fold 0: MSE=2.216004244182793, R2=-2.4942757024646145
Fold 1: MSE=2.4525600803196936, R2=-3.314617798196024
Fold 2: MSE=2.226252183583385, R2=-3.0298159193666088
Fold 3: MSE=2.1635605949376804, R2=-2.621187621214018
Fold 4: MSE=2.0723286504954315, R2=-2.600698919673906
ridge_batch_xavier_momFalse_lr0.01: MSE=2.2261, R2=-2.8121
Fold 0: MSE=12.488196740435404, R2=-18.69184064166517
Fold 1: MSE=12.57336470295772, R2=-21.119443093732325
Fold 2: MSE=12.06121465567199, R2=-20.832420956053085
Fold 3: MSE=12.34291464519496, R2=-19.65854306436406
Fold 4: MSE=12.080632306819368, R2=-19.99026121447626
ridge_batch_xavier_momFalse_lr0.001: MSE=12.3093, R2=-20.0585
Fold 0: MSE=129.05272744320638, R2=-202.49501181026935
Fold 1: MSE=124.47517219261019, R2=-217.98048397890588
Fold 2: MSE=126.46972719603586, R2=-227.92721845743597
Fold 3: MSE=124.4230048286645, R2=-207.2488680622277
Fold 4: MSE=124.20386300309599, R2=-214.80588350581192
ridge_batch_xavier_momFalse_lr0.0001: MSE=125.7249, R2=-214.0915
Fold 0: MSE=0.07781252372607943, R2=0.8773023960954731
Fold 1: MSE=0.08265577360434859, R2=0.8545894656042445
Fold 2: MSE=0.07104918926275264, R2=0.8713914101643787
Fold 3: MSE=0.07592367936874717, R2=0.8729252656336863
Fold 4: MSE=0.07687214391557236, R2=0.8664336153999292
ridge_sto_zeros_momTrue_lr0.01: MSE=0.0769, R2=0.8685
Fold 0: MSE=0.07289688968061112, R2=0.8850535457840976
Fold 1: MSE=0.08624397933285344, R2=0.8482769856678599
Fold 2: MSE=0.06805502230906055, R2=0.8768112551146523
Fold 3: MSE=0.07597336141218954, R2=0.8728421119650851
Fold 4: MSE=0.07877900866642618, R2=0.863120412232699
ridge_sto_zeros_momTrue_lr0.001: MSE=0.0764, R2=0.8692
Fold 0: MSE=0.7482359069362844, R2=-0.1798454611187854
Fold 1: MSE=0.8321832401551996, R2=-0.46400190076731085
Fold 2: MSE=0.782734526179087, R2=-0.41685478289222
Fold 3: MSE=0.7124441963955769, R2=-0.19242978950064638
Fold 4: MSE=0.7266472891850267, R2=-0.26255944419191746
ridge_sto_zeros_momTrue_lr0.0001: MSE=0.7604, R2=-0.3031
Fold 0: MSE=0.07180545253421232, R2=0.8867745633820738
Fold 1: MSE=0.08744658318069146, R2=0.8461613286416783
Fold 2: MSE=0.07329308056477922, R2=0.867329665067882
Fold 3: MSE=0.07609392657680267, R2=0.8726403200288351
Fold 4: MSE=0.0752924783872782, R2=0.8691783055145088
ridge_sto_xavier_momTrue_lr0.01: MSE=0.0768, R2=0.8684
Fold 0: MSE=0.073461687104196, R2=0.8841629527632457
Fold 1: MSE=0.08438351485740411, R2=0.8515499709876074
Fold 2: MSE=0.0685673018811589, R2=0.8758839601792306
Fold 3: MSE=0.07838317697367501, R2=0.8688087632800156
Fold 4: MSE=0.08200583202135861, R2=0.8575137632268542
ridge_sto_xavier_momTrue_lr0.001: MSE=0.0774, R2=0.8676
Fold 0: MSE=0.7751238802378492, R2=-0.22224339065471144
Fold 1: MSE=0.8227910704423586, R2=-0.4474789120210294
Fold 2: MSE=0.7859267001152582, R2=-0.42263304711593475
Fold 3: MSE=0.7186588933706908, R2=-0.20283143196378584
Fold 4: MSE=0.745104094373607, R2=-0.2946283915991095
ridge_sto_xavier_momTrue_lr0.0001: MSE=0.7695, R2=-0.3180
Fold 0: MSE=0.2418394475275894, R2=0.6186588055454034
Fold 1: MSE=0.27433454691452464, R2=0.5173823759607307
Fold 2: MSE=0.24676453121713682, R2=0.5533230046592292
Fold 3: MSE=0.22972179737247905, R2=0.6155107784294725
Fold 4: MSE=0.24163336410938466, R2=0.5801587779535691
ridge_mini_zeros_momTrue_lr0.01: MSE=0.2469, R2=0.5770
Fold 0: MSE=2.2855810635244667, R2=-2.603986949597193
Fold 1: MSE=2.5146670333086325, R2=-3.4238782264758765
Fold 2: MSE=2.353172708607556, R2=-3.2595591425326003
Fold 3: MSE=2.2098746160376117, R2=-2.698704174384809
Fold 4: MSE=2.18395961902804, R2=-2.794659229830995
ridge_mini_zeros_momTrue_lr0.001: MSE=2.3095, R2=-2.9562
Fold 0: MSE=16.11956020283739, R2=-24.41790598959884
Fold 1: MSE=15.841484719485702, R2=-26.86881857411374
Fold 2: MSE=15.645263975771877, R2=-27.32003233828472
Fold 3: MSE=15.7111466256208, R2=-25.29600936941412
Fold 4: MSE=15.372205159275289, R2=-25.709413343668
ridge_mini_zeros_momTrue_lr0.0001: MSE=15.7379, R2=-25.9224
Fold 0: MSE=0.24486824377227867, R2=0.6138828899968136
Fold 1: MSE=0.27618683963751006, R2=0.5141237666348297
Fold 2: MSE=0.24435639743896903, R2=0.5576820507308224
Fold 3: MSE=0.22506135173375727, R2=0.6233110443872496
Fold 4: MSE=0.2402064127539751, R2=0.5826381251375314
```

```
ridge_mini_xavier_momTrue_lr0.01: MSE=0.2461, R2=0.5783
Fold 0: MSE=2.2987405710698976, R2=-2.624737337414864
Fold 1: MSE=2.5232137414578557, R2=-3.4389138536935464
Fold 2: MSE=2.3813357083904876, R2=-3.310537960520641
Fold 3: MSE=2.189390495970462, R2=-2.6644195593885
Fold 4: MSE=2.2068198512044233, R2=-2.834379190890897
ridge_mini_xavier_momTrue_lr0.001: MSE=2.3199, R2=-2.9746
Fold 0: MSE=15.827073646230893, R2=-23.95670260033163
Fold 1: MSE=15.437846544534416, R2=-26.15872610068117
Fold 2: MSE=15.323734551756937, R2=-26.738020829887407
Fold 3: MSE=15.861836971369218, R2=-25.548222326119568
Fold 4: MSE=15.03243064555197, R2=-25.119050553381914
ridge_mini_xavier_momTrue_lr0.0001: MSE=15.4966, R2=-25.5041
Fold 0: MSE=12.690068596592207, R2=-19.010159491383927
Fold 1: MSE=12.506281419150648, R2=-21.001428153915764
Fold 2: MSE=12.27105657319722, R2=-21.212263053920214
Fold 3: MSE=12.313902083691657, R2=-19.609984254030163
Fold 4: MSE=12.0357808923452, R2=-19.912331278216207
ridge_batch_zeros_momTrue_lr0.01: MSE=12.3634, R2=-20.1492
Fold 0: MSE=123.905126959749, R2=-194.378089045993
Fold 1: MSE=123.07520501986163, R2=-215.51761942814477
Fold 2: MSE=123.49709882875909, R2=-222.54636124586392
Fold 3: MSE=123.7328705085613, R2=-206.0937786865106
Fold 4: MSE=122.90005807809369, R2=-212.54050490198915
ridge_batch_zeros_momTrue_lr0.001: MSE=123.4221, R2=-210.2153
Fold 0: MSE=161.82388534550861, R2=-254.16975977171776
Fold 1: MSE=160.9989369369314, R2=-282.2341944945036
Fold 2: MSE=161.5562304765158, R2=-291.4385090997214
Fold 3: MSE=161.81927245879842, R2=-269.8396278212573
Fold 4: MSE=160.93816277292194, R2=-278.63222372678416
ridge_batch_zeros_momTrue_lr0.0001: MSE=161.4273, R2=-275.2629
Fold 0: MSE=12.839364501109504, R2=-19.245574677524353
Fold 1: MSE=12.321675928992795, R2=-20.67666459771536
Fold 2: MSE=12.235936354851681, R2=-21.148690734474947
Fold 3: MSE=11.95406940867808, R2=-19.00772627636302
Fold 4: MSE=11.791999265698317, R2=-19.488757421099518
ridge_batch_xavier_momTrue_lr0.01: MSE=12.2286, R2=-19.9135
Fold 0: MSE=127.00732144342574, R2=-199.26974159453263
Fold 1: MSE=127.19984137840679, R2=-222.77380433732165
Fold 2: MSE=122.61658080248309, R2=-220.9525051743272
Fold 3: MSE=126.64921476198454, R2=-210.97491293086907
Fold 4: MSE=127.77561284586774, R2=-221.01184692630466
ridge_batch_xavier_momTrue_lr0.001: MSE=126.2497, R2=-214.9966
Fold 0: MSE=161.33030552678065, R2=-253.3914652479886
Fold 1: MSE=161.08888228932412, R2=-282.39242907616085
Fold 2: MSE=160.04454084848228, R2=-288.7021475261707
Fold 3: MSE=163.27208039696097, R2=-272.2712168111775
Fold 4: MSE=167.36147165596626, R2=-289.7928093560859
ridge_batch_xavier_momTrue_lr0.0001: MSE=162.6195, R2=-277.3100
d:\AIT\Machine Learning\Assignment 2\App\Code\regression_models.py:55: RuntimeWarning: overflow encountered in square
  return ((ypred - ytrue) ** 2).sum() / ytrue.shape[0]
c:\Users\arpan\miniconda3\envs\ML_class\Lib\site-packages\numpy\_core\_methods.py:49: RuntimeWarning: overflow encountered in reduce
  return umr_sum(a, axis, dtype, out, keepdims, initial, where)
d:\AIT\Machine Learning\Assignment 2\App\Code\regression_models.py:58: RuntimeWarning: overflow encountered in square
  ss_res = ((ytrue - ypred) ** 2).sum()
Fold 0: MSE=inf, R2=-inf
Fold 1: MSE=inf, R2=-inf
Fold 2: MSE=inf, R2=-inf
Fold 3: MSE=inf, R2=-inf
Fold 4: MSE=9.926964398830691e+248, R2=-1.7248234240242579e+249
polynomial_sto_zeros_momFalse_lr0.01: MSE=inf, R2=-inf
Fold 0: MSE=4.03249819276086e+252, R2=-6.358589110190548e+252
Fold 1: MSE=8.502993174758285e+266, R2=-1.4958722513729086e+267
Fold 2: MSE=9.973795142564659e+269, R2=-1.8053910845497434e+270
Fold 3: MSE=6.388305977134662e+275, R2=-1.0692214758881711e+276
Fold 4: MSE=0.1163641622113008 5, R2=0.7978157021267733
polynomial_sto_zeros_momFalse_lr0.001: MSE=127766319188655957381982824003070549590149518285071778787570226015294329347955989190573890146942561893476491125215297216516560287868157203990435905281079071651505948772080401325155100442040090839554
758034668484133040775911374695333793054778849270299150797609860645337528532992.0000, R2=-21384465655502555694122257101639979009981927683650460521320754886435155573955941798928451133392314496450262298675050803104358168441679915307261754949246
4626623704748583885448582527755610807639343829590384611284284012008833477298568487842998110504270322790670473644559730475008.0000
Fold 0: MSE=0.07173497611305757, R2=0.8868856931539071
Fold 1: MSE=0.0719755151940212, R2=0.8733784989071615
Fold 2: MSE=0.061973959337594954, R2=0.8878187970947521
Fold 3: MSE=0.06842535746956555, R2=0.8854753326411586
Fold 4: MSE=0.19465121453963313, R2=0.6617908951176681
polynomial_sto_zeros_momFalse_lr0.0001: MSE=0.0938, R2=0.8391
Fold 0: MSE=inf, R2=-inf
Fold 1: MSE=inf, R2=-inf
Fold 2: MSE=inf, R2=-inf
Fold 3: MSE=inf, R2=-inf
Fold 4: MSE=1.2437001409115874e+248, R2=-2.1609457325738543e+248
polynomial_sto_xavier_momFalse_lr0.01: MSE=inf, R2=-inf
Fold 0: MSE=1.8398449385371586e+277, R2=-2.901134093903201e+277
Fold 1: MSE=4.765788184193698e+279, R2=-8.38411857346797e+279
Fold 2: MSE=1.6227894948061945e+224, R2=-2.9374672771459448e+224
Fold 3: MSE=1.7565138025428402e+254, R2=-2.939906584147652e+254
Fold 4: MSE=0.11876418490406286, R2=0.7936456303985486
polynomial_sto_xavier_momFalse_lr0.001: MSE=95683732671581396901771319740264481571159797761836416190546471165344925929084978683669900277928098091623050016766276902518430471676665726567436435216296307239395805790806775363021762149518345662 22212
915880780020811768381861784297889945700355900949364988687659045891798495242328 4736.0000, R2=-1682625982881400431664281369168585284893502779122606703701024254705438200433108263613599011990586413580877008741310094442242922688617041603865431638
814359147883382873039801229104706216056226769469824176186668693615020573128005988268151217816742327605837169610300905877984912605 1840.0000
Fold 0: MSE=0.06918062883106145, R2=0.8909134804050218
Fold 1: MSE=0.07343222009496314, R2=0.8708158196306137
Fold 2: MSE=0.06473777365790472, R2=0.8828159213970714
Fold 3: MSE=0.07368625085583998, R2=0.8766700872269656
Fold 4: MSE=0.3736916488659617, R2=0.35070573094612767
polynomial_sto_xavier_momFalse_lr0.0001: MSE=0.1309, R2=0.7744
Fold 0: MSE=0.06800688483504203, R2=0.8927642824226493
Fold 1: MSE=0.07407802158779701, R2=0.8696797061313191
Fold 2: MSE=0.05893728964984875, R2=0.8933155777109716
Fold 3: MSE=0.07402948108213563, R2=0.8760956170459148
Fold 4: MSE=0.07089489524118071, R2=0.8768191654136102
polynomial_mini_zeros_momFalse_lr0.01: MSE=0.0692, R2=0.8817
Fold 0: MSE=0.37059298639485855, R2=0.4156355651110676
Fold 1: MSE=0.3978490573747599, R2=0.30009191712812433
Fold 2: MSE=0.39033512866631537, R2=0.293440910698779
Fold 3: MSE=0.3451488868711521, R2=0.42231852459412433
Fold 4: MSE=0.6235300413418782, R2=-0.08339183831057895
polynomial_mini_zeros_momFalse_lr0.001: MSE=0.4255, R2=0.2696
Fold 0: MSE=2.2334425101424142, R2=-2.5217729914232336
Fold 1: MSE=2.469801501567872, R2=-3.34494942741095
Fold 2: MSE=2.3395836497235494, R2=-3.234961118003395
Fold 3: MSE=2.202499090261154, R2=-2.68635963330556
Fold 4: MSE=2.292658834019027, R2=-2.983525578753922
polynomial_mini_zeros_momFalse_lr0.0001: MSE=2.3076, R2=-2.9543
Fold 0: MSE=0.06709437109918219, R2=0.8942031671105962
Fold 1: MSE=0.06964554864523272, R2=0.8774774464604845
Fold 2: MSE=0.058727181074865645, R2=0.8936959024947073
Fold 3: MSE=0.06584897424427309, R2=0.8897874684162147
Fold 4: MSE=0.07717982203502155, R2=0.8658990205266484
polynomial_mini_xavier_momFalse_lr0.01: MSE=0.0677, R2=0.8842
Fold 0: MSE=0.37639447928689834, R2=0.4064875611287234
Fold 1: MSE=0.3990168449845489, R2=0.2980375098799024
Fold 2: MSE=0.3977368346501222, R2=0.28004282721840457
Fold 3: MSE=0.351141478250631, R2=0.4122886239880874
Fold 4: MSE=0.7283817215585378, R2=-0.26557304378291646
polynomial_mini_xavier_momFalse_lr0.001: MSE=0.4505, R2=0.2263
Fold 0: MSE=2.2956543495538075, R2=-2.619870871619771
Fold 1: MSE=2.466926642623972, R2=-3.339891888692047
Fold 2: MSE=2.3120898123711098, R2=-3.1851935740277604
Fold 3: MSE=2.2219656772418053, R2=-2.7189411861248143
Fold 4: MSE=2.266233031747708, R2=-2.937610391668559
polynomial_mini_xavier_momFalse_lr0.0001: MSE=2.3126, R2=-2.9603
Fold 0: MSE=2.153737708398207, R2=-2.396091575047043
Fold 1: MSE=2.3817540523928087, R2=-3.1900536944399684
Fold 2: MSE=2.264843607912587, R2=-3.0996715885758936
Fold 3: MSE=2.121499878966650 6, R2=-2.5507898961075113
Fold 4: MSE=2.222403539233 14, R2=-2.8611725218943858
```

```
polynomial_batch_zeros_momFalse_lr0.01: MSE=2.2288, R2=-2.8196
Fold 0: MSE=11.96370100771746, R2=-17.864796785727723
Fold 1: MSE=12.039988646726696, R2=-20.18111181948032
Fold 2: MSE=11.721454786878285, R2=-20.217409890315256
Fold 3: MSE=11.721081314235798, R2=-18.617770198655414
Fold 4: MSE=11.700333841069071, R2=-19.329487512171188
polynomial_batch_zeros_momFalse_lr0.001: MSE=11.8293, R2=-19.2421
Fold 0: MSE=124.42739764095667, R2=-195.20162436017134
Fold 1: MSE=123.62752022103248, R2=-216.48926901842736
Fold 2: MSE=124.02954824980313, R2=-223.51016632104958
Fold 3: MSE=124.2751092955852, R2=-207.00133283031752
Fold 4: MSE=123.52635934374808, R2=-213.62871177983791
polynomial_batch_zeros_momFalse_lr0.0001: MSE=123.9772, R2=-211.1662
Fold 0: MSE=2.1404901344170204, R2=-2.3752023208858293
Fold 1: MSE=2.381643894045017, R2=-3.189859900546063
Fold 2: MSE=2.272872409437796, R2=-3.114204799340743
Fold 3: MSE=2.11265186049406, R2=-2.5359808193290814
Fold 4: MSE=2.099766933344752, R2=-2.6483733053895286
polynomial_batch_xavier_momFalse_lr0.01: MSE=2.2015, R2=-2.7727
Fold 0: MSE=11.76665144527533, R2=-17.554081903285457
Fold 1: MSE=11.832899944907691, R2=-19.81679511798809
Fold 2: MSE=11.81711938613582, R2=-20.39057568341392
Fold 3: MSE=11.803188085762166, R2=-18.755193677972244
Fold 4: MSE=11.609678613455305, R2=-19.171972834152506
polynomial_batch_xavier_momFalse_lr0.001: MSE=11.7659, R2=-19.1377
Fold 0: MSE=125.73037527066533, R2=-197.2562066491258
Fold 1: MSE=120.3812542204187, R2=-210.77833978316835
Fold 2: MSE=123.01062816308026, R2=-221.66578390277874
Fold 3: MSE=121.01567504563697, R2=-201.54596310982512
Fold 4: MSE=120.35264141336538, R2=-208.11433416384213
polynomial_batch_xavier_momFalse_lr0.0001: MSE=122.0981, R2=-207.8721
Fold 0: MSE=8.499814917689657e+257, R2=-1.3402815820545555e+258
Fold 1: MSE=9.924890672684907e+262, R2=-1.7460167555175288e+263
Fold 2: MSE=4.6259946598690046e+226, R2=-8.37367260578671e+226
Fold 3: MSE=9.346501387336409e+248, R2=-1.5643395985614694e+249
Fold 4: MSE=0.12562585702936097, R2=0.7817233827363302
polynomial_sto_zeros_momTrue_lr0.01: MSE=1984995134166835643426668245741134895356100672797711709980086614073252330523875126367527832267918905270925569867987942046797805306199649495042898961369800592059629045209666299594603270185552564984720 3
494475546254600686711459858483000815496355944598995064863064064.0000, R2=-3492060316666729876825423997599900780369894475842601621102389404338866289242821756927956968833076144376383998901901750988617804930861589501937480373805515183244390 9470
817037501588462715707812757090188255124820452582553108683149443614180961549801067243635006767104.0000
Fold 0: MSE=0.0700494145691825 5, R2=0.8895435476066829
Fold 1: MSE=0.07193841996619674, R2=0.8734437579527946
Fold 2: MSE=0.06202937118204076 4, R2=0.8877184942025119
Fold 3: MSE=0.07889853497459924, R2=0.8679461999583287
Fold 4: MSE=0.3878138578032502, R2=0.326168202860634 5
polynomial_sto_zeros_momTrue_lr0.001: MSE=0.1341, R2=0.7690
Fold 0: MSE=0.9294401349881986, R2=-0.46557484675864025
Fold 1: MSE=1.0139922742171101, R2=-0.7838458469079064
Fold 2: MSE=0.9959123103541209, R2=-0.8027352481237415
Fold 3: MSE=0.8899851568329977, R2=-0.48958306993047174
Fold 4: MSE=1.1153310910875927, R2=-0.9379027809115679
polynomial_sto_zeros_momTrue_lr0.0001: MSE=0.9889, R2=-0.6959
Fold 0: MSE=9.568702224004778e+257, R2=-1.5088276014466415e+258
Fold 1: MSE=2.691428392030019e+257, R2=-4.734842149640276e+257
Fold 2: MSE=8.256399083990884e+257, R2=-1.4945192961812606e+258
Fold 3: MSE=5.906519041115751e+262, R2=-9.885839891056681e+262
Fold 4: MSE=0.11372303563761937, R2=0.8024046950928763
polynomial_sto_xavier_momTrue_lr0.01: MSE=1181344841282550228462218461041817538297828128150620699598071214724195238893855671272266846373661519137514164324479371628145725837414822517234663483630915651346020794460016613923314066407198018055113
63132493105459935700594864438984715028979002132204543630418903 04.0000, R2=-1977237514833587824298675314853370034298024432026164179868857640528326752902379207460562122861599512651044183390985541732388644017072334552768040826581280685958359689
9995889236577189535328881167619845760807722443885345772142560829851706852950282064567622348832768.0000
Fold 0: MSE=0.08565953846890469, R2=0.8649289392193346
Fold 1: MSE=0.07289561470051702, R2=0.8717598320542292
Fold 2: MSE=0.062237879558118304, R2=0.8873410659940482
Fold 3: MSE=0.0685920086337864, R2=0.8851964057951301
Fold 4: MSE=0.25482539189982445, R2=0.557237452129121
polynomial_sto_xavier_momTrue_lr0.001: MSE=0.1088, R2=0.8133
Fold 0: MSE=0.9206322192859262, R2=-0.45168620646898394
Fold 1: MSE=0.9958127652878517, R2=-0.7518639054998011
Fold 2: MSE=0.9956018164026408, R2=-0.8021732123050827
Fold 3: MSE=0.8761122404972762, R2=-0.466363737399367
Fold 4: MSE=1.011447290171635, R2=-0.7574032787499378
polynomial_sto_xavier_momTrue_lr0.0001: MSE=0.9599, R2=-0.6459
Fold 0: MSE=0.3692662790631452, R2=0.41772756525306065
Fold 1: MSE=0.399676176271466 5, R2=0.2968775943580485
Fold 2: MSE=0.3909081397277462 4, R2=0.2924036835982937
Fold 3: MSE=0.3450815617052352, R2=0.42243120785244626
Fold 4: MSE=0.8453941316262913, R2=-0.4688836810308614
polynomial_mini_zeros_momTrue_lr0.01: MSE=0.4701, R2=0.1921
Fold 0: MSE=2.23340160129368, R2=-2.5217084848698175
Fold 1: MSE=2.4712686156460677, R2=-3.347530418826639
Fold 2: MSE=2.33966542636529, R2=-3.2351091447252314
Fold 3: MSE=2.641503103772076, R2=-3.421128006841598
Fold 4: MSE=2.3672576145109554, R2=-3.1131419638104436
polynomial_mini_zeros_momTrue_lr0.001: MSE=2.4106, R2=-3.1277
Fold 0: MSE=15.575213130969846, R2=-23.559559823553492
Fold 1: MSE=15.565853179932715, R2=-26.38391924143583
Fold 2: MSE=15.283818078847876, R2=-26.665766644508018
Fold 3: MSE=15.300994458845711, R2=-24.60953081521266
Fold 4: MSE=15.234950251657557, R2=-25.47093109450305
polynomial_mini_zeros_momTrue_lr0.0001: MSE=15.3922, R2=-25.3379
Fold 0: MSE=0.8452643133390539, R2=-0.33284336436381756
Fold 1: MSE=1.6236023024321478, R2=-1.856290228108357
Fold 2: MSE=1.6914469622562507, R2=-2.0617465287751515
Fold 3: MSE=0.3749079675272851, R2=0.37251025264516957
Fold 4: MSE=1.122972574047794, R2=-0.95117996039414
polynomial_mini_xavier_momTrue_lr0.01: MSE=1.1316, R2=-0.9659
Fold 0: MSE=2.2308044169710146, R2=-2.5176131506224504
Fold 1: MSE=2.5344915295756696, R2=-3.4587540793124276
Fold 2: MSE=2.3764611386088044, R2=-3.301714333506979
Fold 3: MSE=2.2039341385089934, R2=-2.688761497604198
Fold 4: MSE=2.671309047040552, R2=-3.6414354197605343
polynomial_mini_xavier_momTrue_lr0.001: MSE=2.4034, R2=-3.1217
Fold 0: MSE=15.516436718932951, R2=-23.4668790495894
Fold 1: MSE=15.489110254765631, R2=-26.248910768669546
Fold 2: MSE=15.384255307855296, R2=-26.847571539450197
Fold 3: MSE=14.758799664235504, R2=-23.7020503022457
Fold 4: MSE=15.64734799511828, R2=-26.187477723822674
polynomial_mini_xavier_momTrue_lr0.0001: MSE=15.3592, R2=-25.2906
Fold 0: MSE=12.157751107499395, R2=-18.170782006879183
Fold 1: MSE=12.23863959877718, R2=-20.53058457663042
Fold 2: MSE=11.91818907568975, R2=-20.573525417021493
Fold 3: MSE=11.914698193613717, R2=-18.94183001399033
Fold 4: MSE=11.897811932422016, R2=-19.67260835356081
polynomial_batch_zeros_momTrue_lr0.01: MSE=12.0254, R2=-19.5779
Fold 0: MSE=123.87577640294242, R2=-194.331808025651
Fold 1: MSE=123.07751222193178, R2=-215.52167832773947
Fold 2: MSE=123.47682015495923, R2=-222.50965411847514
Fold 3: MSE=123.72164618260408, R2=-206.0749923440768
Fold 4: MSE=122.97540899728571, R2=-212.67142813815937
polynomial_batch_zeros_momTrue_lr0.001: MSE=123.4254, R2=-210.2219
Fold 0: MSE=161.82519224178557, R2=-254.17182053307158
Fold 1: MSE=161.0027813581848, R2=-282.240957716533
Fold 2: MSE=161.55742604570744, R2=-291.4406732407817
Fold 3: MSE=161.82232662417556, R2=-269.8447396290258
Fold 4: MSE=160.95104081891543, R2=-278.6545995050108
polynomial_batch_zeros_momTrue_lr0.0001: MSE=161.4318, R2=-275.2706
Fold 0: MSE=12.081663254000077, R2=-18.05080392541418
Fold 1: MSE=12.108720524429682, R2=-20.302027015491372
Fold 2: MSE=12.197001821264728, R2=-21.078213991355362
Fold 3: MSE=12.038938790966506, R2=-19.149773583603693
Fold 4: MSE=11.917161707161068, R2=-19.706228847579546
polynomial_batch_xavier_momTrue_lr0.01: MSE=12.0687, R2=-19.6574
Fold 0: MSE=124.15550257923208, R2=-194.77289038536134
Fold 1: MSE=124.48474837190135, R2=-217.99733068286224
Fold 2: MSE=121.90757729099728, R2=-219.66911344604893
Fold 3: MSE=124.28951043119723, R2=-207.02543625230243
Fold 4: MSE=123.52365554424729, R2=-213.6240138918175
polynomial_batch_xavier_momTrue_lr0.001: MSE=123.6722, R2=-210.6178
Fold 0: MSE=163.21806779838056, R2=-256.3681571270755
Fold 1: MSE=158.63984667617564, R2=-278.0840116271053
Fold 2: MSE=161.38246044930068, R2=-291.1239619755707
Fold 3: MSE=159.50766123559706, R2=-265.9706453827241
Fold 4: MSE=154.94769567558262, R2=-268.22370652533783
polynomial_batch_xavier_momTrue_lr0.0001: MSE=159.5391, R2=-271.9541
```

Out[16]:

	model_type	method	initialise_method	use_momentum	lr	cv_mse	cv_r2
117	polynomial	mini	xavier	False	0.010	0.067699	0.884213
114	polynomial	mini	zeros	False	0.010	0.069189	0.881735
18	normal	sto	zeros	True	0.010	0.073174	0.874900
1	normal	sto	zeros	False	0.001	0.073775	0.873897

3	normal	sto	xavier	False	0.010	0.075207	0.871387
76	ridge	sto	xavier	False	0.001	0.075224	0.871257
21	normal	sto	xavier	True	0.010	0.075398	0.870985
73	ridge	sto	zeros	False	0.001	0.075787	0.870316
6	normal	mini	zeros	False	0.010	0.075967	0.870168
4	normal	sto	xavier	False	0.001	0.076276	0.869604

In [17]:

```
best = results_df.iloc[0]

print("Best model:")
print(best)

# Choose regularization
if best["model_type"] == "lasso":
    regularization = Lasso(0.01)
elif best["model_type"] == "ridge":
    regularization = Ridge(0.01)
else:
    regularization = NoRegularization()

# Create final model
final_model = LinearRegression(
    regularization=regularization,
    lr=float(best["lr"]),
    method=best["method"],
    num_epochs=500,
    cv=KFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    ),
    initialise_method=best["initialise_method"],
    use_momentum=bool(best["use_momentum"]),
    momentum=0.9
)

# Select correct features
if best["model_type"] == "polynomial":
    Xtr_final = X_train_poly
    Xte_final = X_test_poly
else:
    Xtr_final = X_train_np
    Xte_final = X_test_np

# Train final model
final_model.fit(Xtr_final, y_train_np)

# Predict
y_pred_log = final_model.predict(Xte_final)

# Convert back from log price
y_pred_price = np.exp(y_pred_log)
y_test_price = np.exp(y_test_np)

# Test metrics
test_mse = final_model.mse(y_test_price, y_pred_price)
test_r2 = final_model.r2(y_test_price, y_pred_price)

print("\nFinal Test Results")
print("------------------")
print("Test MSE:", test_mse)
print("Test R2:", test_r2)
```

```
Best model:
model_type           polynomial
method                     mini
initialise_method        xavier
use_momentum              False
lr                         0.01
cv_mse                 0.067699
cv_r2                  0.884213
Name: 117, dtype: object
Fold 0: MSE=0.06761880985474861, R2=0.8933762130982936
Fold 1: MSE=0.0903034858267085, R2=0.8411353791845658
Fold 2: MSE=0.05925494304132874, R2=0.8927405823428519
Fold 3: MSE=0.0679796945282486, R2=0.8862212461737431
Fold 4: MSE=0.06525332135727163, R2=0.88662147596131

Final Test Results
------------------
Test MSE: 40630302783.24909
Test R2: 0.8535863179998685
```

In [18]:

```
from car_model_v2 import CarPriceModelV2
import pickle

medians = {col: X_train[col].median() for col in num_features}

# modes: needs the categorical columns BEFORE get_dummies encoded them. If you no longer
# have that DataFrame in memory, recompute it from the cleaned df, filtered to the same
# training rows:
X_train_raw_cats = df.loc[X_train.index, cat_features]
modes = {col: X_train_raw_cats[col].mode()[0] for col in cat_features}

car_model_v2 = CarPriceModelV2(
    model=final_model,
    scaler=scaler,
    medians=medians,
    modes=modes,
    columns=feature_names,
    num_features=num_features,
    cat_features=cat_features,
)

pickle.dump(car_model_v2, open('Car_Price_Model_V2.pkl', 'wb'))
print("Saved Car_Price_Model_V2.pkl")
```

```
Saved Car_Price_Model_V2.pkl
```

In [19]:

```
best_per_type = results_df.sort_values('cv_mse').groupby('model_type').first().reset_index()
best_per_type = best_per_type[['model_type', 'method', 'initialise_method', 'use_momentum', 'lr', 'cv_mse', 'cv_r2']]
best_per_type
```

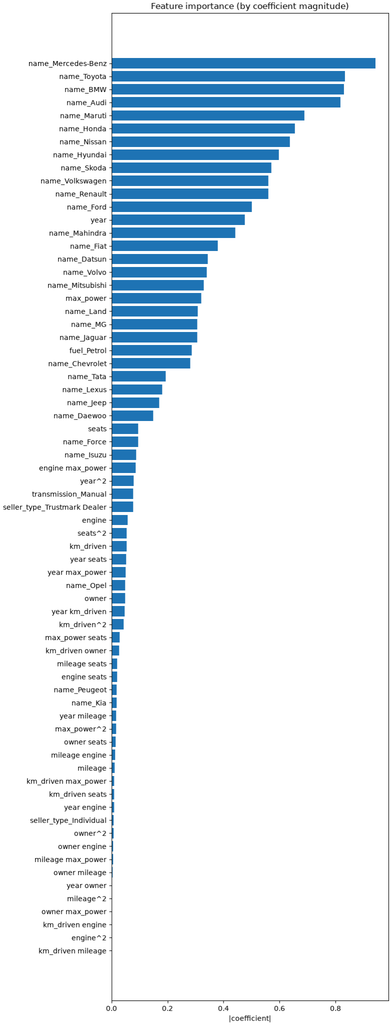
Out[19]:

	model_type	method	initialise_method	use_momentum	lr	cv_mse	cv_r2
0	lasso	mini	xavier	False	0.010	0.083041	0.857838
1	normal	sto	zeros	True	0.010	0.073174	0.874900
2	polynomial	mini	xavier	False	0.010	0.067699	0.884213
3	ridge	sto	xavier	False	0.001	0.075224	0.871257

In [20]:

```
# Feature importance
if best["model_type"] == "polynomial":
    names_to_use = poly_feature_names
else:
    names_to_use = feature_names

final_model.plot_feature_importance(feature_names=names_to_use)
```



Across the 144 configurations, model type had the biggest effect on performance. Normal linear regression and Lasso had CV MSE values around 0.07–0.09, while Ridge was much worse at around 0.6–2.4. This is mainly because the implementation applies regularization to the bias as well as the other parameters. Since log(selling_price) needs a large bias (around 13), Ridge's L2 penalty makes it harder for the model to reach the correct prediction scale. Lasso was less affected and therefore performed much closer to normal linear regression.

Stochastic updates with lr=0.01–0.001 worked best for normal linear regression, while full-batch updates with very small learning rates were too slow to converge within 500 epochs. Higher learning rates sometimes caused divergence. Xavier initialization and momentum only gave small and inconsistent improvements, which is expected for a simple linear model.

Polynomial features caused stability problems because the expanded features had values up to around 1400, compared with roughly -3 to 3 for the standardized original features. This caused several configurations to diverge and produce infinite loss. Applying a second StandardScaler after polynomial expansion fixed most of the problem, although the polynomial models still needed smaller learning rates to remain stable.